

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	CSA0305	Course Title:	Data Structures
CO Assessed:	CO4 - Articulation (BL3)	Assessment:	Assessment Tool 3 - Comparative Analysis
Weightage:	30% of CIA	Total Marks:	100
CO4 Statement:	Apply hashing strategies for efficient data storage and sorting algorithms for arrangement of data.		
Student Name:	G.koushik	Reg. No.:	192524238
Section / Batch:		Date:	26/08/2025

Code of Conduct

I, G.koushik (192524238) (Name / Reg No)

certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: G.koushik

Instructions

- This test contains 5 **Comparative Analysis** questions. Total: 100 Marks.
- When a student answer using **comparative analysis** should be based on four requirements: time complexity, space complexity advantages & limitations, real-world scenarios and operations.
- No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

Section / Topic	Focus	Q Nos	Marks
Hashing	Linear Probing & Separate Chaining	1	20
Sorting	Merge Sort & Quick Sort	2	20
Hashing	Quadratic Probing and Double Hashing	3	20
Sorting Algorithms	Complexity Analysis	4	20
Quick Sort	Recursive and Iterative implementations	5	20
GRAND TOTAL:			100 Marks

Annexure A - Comparative Analysis

1. Compare Linear Probing and Separate Chaining for collision handling in hashing based on: (20 Marks)
 - a) Clustering effect
 - b) Memory usage
 - c) Performance under high load factor
 - d) Which method is more suitable for large-scale applications and why?
2. Compare quick sort and merge sort for sorting a large linked list of 10 million nodes. Analyze memory requirements (in-place vs. extra space), cache behavior, and time complexity. Which algorithm is preferable for linked lists and why? How does the answer change for arrays? (20 Marks)
3. Compare Quadratic Probing and Double Hashing in terms of: (20 Marks)
 - a) Clustering issues
 - b) Collision resolution efficiency
 - c) Suitability for dense hash tables
4. Compare Best-case and Worst-case time complexity of sorting algorithms based on the following: (20 Marks)
 - a) Practical significance
 - b) Impact on algorithm selection
5. Compare Recursive and Iterative implementations of Quick Sort in terms of: (20 Marks)
 - a) Space complexity (stack usage)
 - b) Ease of implementation
 - c) Risk of stack overflow

Rubrics for Calculation

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Concept Understanding	20	Demonstrates complete and accurate understanding of all data structures with advanced insights	Shows clear understanding with minor gaps	Basic understanding; some misconceptions present	Limited or incorrect understanding
Comparative Analysis Depth	20	Thorough, multi-dimensional comparison with strong justification and critical evaluation	Good comparison with relevant factors considered	Limited comparison; lacks depth or justification	Minimal or incorrect comparison
Use of Parameters (Time/Space/Operations)	30	All parameters analysed accurately with complexity discussion	Most parameters covered correctly	Few parameters considered; some inaccuracies	Parameters missing or incorrect
Critical Thinking	20	Strong justification of why one structure is better in given contexts	Some justification present	Limited reasoning	No critical reasoning
Clarity	10	Exceptionally well-structured, logical flow, clear tables/diagrams	Well-organized with minor issues	Some organization issues; lacks clarity	Poor structure and readability

Faculty In-charge

Signature: _____

Date: _____

Course Coordinator

Signature: _____

Date: _____

Program Director

Signature: _____

Date: _____

1) Linear Probing vs separate chaining:

Factor	<u>Linear Probing</u>	Separate
Clustering	Causes Primary Clustering because positions	Doesn't cause primary because collision
High load	Performance degrades as table will be full	Handles high load factors shared
Avg Search	$O(1)$ load factor is slow	$O(1)$ with good hash function
Worst Case	$O(n)$	$O(n)$

* Suitable for large scale applications.

Conclusion

Linear probing is memory efficient & cache-friendly with separate chaining is more flexible & suitable for frequently changing datasets

2) Quick Sort vs Merge Sort

Factor	Quick sort	Merge Sort
Avg time	$O(n \log n)$	$O(n \log n)$
worst time	$O(n^2)$	$O(n \log n)$
Extra time	$O(\log n)$ avg recursion	$O(\log n)$ recursion merging can be done using temp
Stability	Usually not suitable	Stable
Linked List	Moderate	Excellent

Quick Sort: It divides list around a pivot. Its avg complexity is $O(n \log n)$, but poor pivot

a) Clustering:

Quadratic Probing reduces primary clustering, but keys with the same initial value follow the same frequency

b) Collision Resolution:

Double hashing generally provides better contribution of collisions than quadratic probing

c) Dense hash Tables:

Double hashing is more suitable for dense hash tables because it distributes collisions.

Conclusion:

Double hashing is generally more efficient when the hash table has a high load factor

Quadratic Probing & Double Hashing

<u>Factor</u>	<u>Quadratic Probing</u>	<u>Double Hashing</u>
Clustering	Avoid primary clustering	Greatly reduces clustering
Collision	uses quadratic jumps to find an empty position	uses 2 nd hash function to determine
Prime tables	Performance decreases as load factor $\uparrow$	Performs better high load factors

Ex:

Quick sort best : $O(n \log n)$

Quick sort worst case : $O(n^2)$

Merge sort best case : $O(n \log n)$

b) Impact on Algorithm:

- Input size
- Expected data arrangement
- Requirement
- Available memory
- Need for predictable performance

4) Best Case vs Worst Case Time Complexity

a) Practical Significance:

- Best-case complexity represents the minimum time an algorithm requires for favourable input

- Worst case complexity represents the max time required for unfavourable input.

Merge Sort.

Merge sort divides the linked list into 2 boxes & merges the sorted halves. It always provides $O(n \log n)$ time complexity.

5) Recursive vs Iterative Quick Sort

<u>Factor</u>	<u>Recursive sort</u>	<u>Iterative Sort</u>
Stack usage	Use function call stack	Uses an explicit stack
Avg space	$O(\log n)$	$O(\log n)$ proper
Worst case	$O(n)$	Can be controlled using explicit stack
Implementation	Simple & Easy to understand	More complex
Stack overflow	Higher risk for unbalanced partition	Lower risk because recursive

a) Space Complexity

Recursive Quick sort requires stack space for recursive function calls. Balanced partition give approximately $O(\log n)$ stack usage.

b) Ease of Implementation

Requires quick sort is easier because divide & conquer structure naturally fits recursion iterative quick sort.